

Universidad del Valle de Guatemala
Facultad de Ingeniería
Análisis y diseño de algoritmos



Parcial 2

Dilary Sarahí Cruz López - 231010

Problema 1

Inciso a

1. **[30 puntos]** Considere el problema de “hacer sencillo”: dado un monto m , buscamos alcanzar dicho monto con la menor cantidad de monedas posible. Las monedas tienen las siguientes denominaciones: $\{1, 5, 10, 25\}$. Podemos tomar todas las monedas que necesitemos de cada denominación.

- ☒ Subestructura óptima
- ☒ Subproblemas traslapados
- ☒ *Greedy-choice property*
- ☐ Se puede modelar como una matroide ponderada

Inciso b

Subestructura óptima

Sea $OPT(m)$ el número mínimo de monedas para el monto m con denominaciones $D = \{1, 5, 10, 25\}$.

Decisiones

Para cualquier solución, se elige alguna $c \in D$ con $c \leq m$.

Subproblema generado: resolver $OPT(m - c)$.

Proposición

si OPT es una solución óptima para m que contiene al menos una moneda c , entonces $OPT \setminus \{c\}$ es óptima para $m - c$.

Demostración

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos



supóngase que $OPT \setminus \{c\}$ no es óptima para $m - c$. Entonces existe S^* con $|S^*| < |OPT| - 1$ que representa $m - c$. Pero entonces $S^* \cup \{c\}$ representa m con $|S^*| + 1 < |OPT|$ monedas, contradiciendo que OPT es óptima.

Subproblemas traslapados

La recursión $T(m) = 1 + \min\{T(m-c) : c \in D, c \leq m\}$ genera, por ejemplo, $T(m-1)$ y $T(m-6)$.

Ambas requieren computar $T(m-6)$ (la primera como $T((m-1)-5)$, la segunda directamente).

$T(k)$ para $k < m$ puede ser requerido por hasta cuatro ramas distintas de la recursión. Sin memoización, el número de llamadas es exponencial en m .

Greedy-choice property

Criterio: en cada paso, elegir $c^* = \max\{c \in D : c \leq m\}$.

Teorema: para $D = \{1, 5, 10, 25\}$, siempre existe una solución óptima que incluye una moneda de denominación c^* .

Demostración por intercambio

Sea OPT una solución óptima para m que no usa ninguna moneda c^* . Analizamos por casos:

- $c^* = 25$ ($m \geq 25$): OPT cubre m solo con $\{1, 5, 10\}$. La menor cantidad de monedas de $\{1, 5, 10\}$ para sumar 25 es 3 (p.ej., $10+10+5$). Sustituir ese subconjunto de OPT por una moneda de 25 reduce la cuenta total en al menos 2. Entonces existe solución con menos monedas que OPT que incluye $c^* = 25$, contradicción.
- $c^* = 10$ ($10 \leq m < 25$): OPT usa solo $\{1, 5\}$. Para sumar 10 con $\{1, 5\}$ se requieren al menos 2 monedas ($5+5$). Una moneda de 10 las reemplaza. Contradicción.
- $c^* = 5$ ($5 \leq m < 10$): OPT usa solo $\{1\}$. Para sumar 5 se necesitan exactamente 5 monedas de 1. Una moneda de 5 las reemplaza. Contradicción.

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos



OPT puede transformarse para incluir c^* sin incrementar el número de monedas. La elección greedy en mayor moneda posible, siempre pertenece a alguna solución óptima.

Inciso c

ALGORITMO GREEDY_COINS(m)

$D \leftarrow [25, 10, 5, 1]$

resultado $\leftarrow \{\}$

total $\leftarrow 0$

PARA cada c EN D HACER

$k \leftarrow m \text{ DIV } c$

resultado[c] $\leftarrow k$

$m \leftarrow m - (k * c)$

total $\leftarrow$ total + k

FIN PARA

RETORNAR (total, resultado)

FIN ALGORITMO

$O(|D|) = O(1)$ — $|D| = 4$ es constante.

Problema 2

Inciso a

2. **[30 puntos]** *Knapsack problem* fraccionado: el problema visto en clase de un ladrón con una bolsa que aguanta W unidades u de peso, y busca robar el máximo valor que pueda de una selección de n artículos, donde el artículo i tiene precio p_i y hay w_i u 's disponibles del mismo. No es necesario robar el artículo completo; es posible tomar porciones del mismo (a nivel de u 's).

- ☒ Subestructura óptima
- ☐ Subproblemas traslapados
- ☒ *Greedy-choice property*
- ☐ Se puede modelar como una matroide ponderada

Inciso b

Subestructura óptima

Sean n artículos con pesos $w_1, \dots, w_n$, precios $p_1, \dots, p_n$, capacidad W . Se toma $x_i \in [0, w_i]$ de cada artículo.

Decisión

Cuánto tomar del artículo i^* de mayor densidad.

Subproblema

knapsack fraccionado con artículos $\{1, \dots, n\} \setminus \{i^*\}$ y capacidad $W - x_{i^*}$.

Proposición

Si $OPT = (x_1, \dots, x_n)$ es óptima, entonces la subsolución $(x_1, \dots, x_n) \setminus \{x_{i^*}\}$ es óptima para el subproblema resultante.

Demostración

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos



Supóngase que existe $S^* = (y_1, \dots, y_n) \setminus \{i^*\}$ con $\text{valor}(S^*) > \text{valor}(\text{OPT} \setminus \{x_{i^*}\})$ y $\sum_{j \neq i^*} y_j w_j \leq W - x_{i^*}$. Vamos viendo que $S^* \cup \{x_{i^*}\}$ es factible y tiene valor $\text{valor}(S^*) + x_{i^*} \cdot \rho_{i^*} > \text{valor}(\text{OPT})$, contradiciendo la optimalidad de OPT.

Greedy-choice property

Sea $i^* = \text{argmax} \{\rho_i = p_i/w_i\}$ (mayor densidad).

Teorema

Existe una solución óptima que toma $q^* = \min(w_{i^*}, W)$ del artículo i^* .

Demostración por intercambio

Sea $\text{OPT} = (x_1, \dots, x_n)$ óptima con $x_{i^*} < q^*$. Como $x_{i^*} < \min(w_{i^*}, W)$, existe $j \neq i^*$ con $x_j > 0$ (si no, podría tomarse más de i^* para mayor valor, contradiciendo optimalidad).

Sea $\delta = \min(x_j, q^* - x_{i^*}) > 0$. Construir OPT' :

- $x'_{i^*} = x_{i^*} + \delta$
- $x'_j = x_j - \delta$
- resto sin cambio.

Factibilidad

$$\sum x'_k = \sum x_k + \delta - \delta = \sum x_k \leq W.$$

Cambio en valor

$$\Delta = \delta \cdot \rho_{i^*} - \delta \cdot \rho_j = \delta(\rho_{i^*} - \rho_j) \geq 0$$

Si $\rho_{i^*} > \rho_j$: $\text{valor}(\text{OPT}') > \text{valor}(\text{OPT})$, contradiciendo optimalidad de OPT.

Si $\rho_{i^*} = \rho_j$: $\text{valor}(\text{OPT}') = \text{valor}(\text{OPT})$ y OPT' también es óptima pero usa más de i^* .

hasta $x_{i^*} = q^*$, se obtiene una solución óptima que agota i^* .

ALGORITMO FRACTIONAL_KNAPSACK(artículos[1..n], W)

PARA $i \leftarrow 1$ HASTA n HACER

$\rho[i] \leftarrow \text{precio}[i] / \text{peso}[i]$

FIN PARA

ordenar artículos por ρ en orden decreciente

valor $\leftarrow 0$

PARA $i \leftarrow 1$ HASTA n HACER

SI $W = 0$ ENTONCES

TERMINAR

FIN SI

cantidad $\leftarrow \text{MIN}(\text{peso}[i], W)$

valor $\leftarrow \text{valor} + \text{cantidad} * \rho[i]$

$W \leftarrow W - \text{cantidad}$

FIN PARA

RETORNAR valor

FIN ALGORITMO

$O(n \log n)$ por ordenamiento; el barrido es $O(n)$. Total: $O(n \log n)$.

Problema 3

Inciso a

3. **[40 puntos]** Dado un número entero positivo n y el teclado de un Nokia 3230, que tiene dígitos del 0 al 9 asociados con cada tecla; se desea contar la cantidad total de combinaciones de n dígitos que existen, bajo las siguientes restricciones:
- Podemos comenzar con cualquier dígito.
 - El siguiente teclazo debe ser arriba, abajo, a la izquierda o a la derecha del teclazo más reciente.
 - El teclado también tiene las teclas * y # que no es permitido presionar.

- ☒ Subestructura óptima
- ☒ Subproblemas traslapados
- ☐ Greedy-choice property
- ☐ Se puede modelar como una matroide ponderada

Inciso b

Teclado:

[' 1 ' , ' 2 ' , ' 3 '],
[' 4 ' , ' 5 ' , ' 6 '],
[' 7 ' , ' 8 ' , ' 9 '],
[' * ' , ' 0 ' , ' # ']

Para $n = 2$, el total de combinaciones posibles son 36:

[00, 08, 11, 12, 14, 21, 22, 23, 25, 32, 33, 36, 41, 44, 45, 47, . . . , 96, 98, 99]

Dígitos válidos: $D = \{0,1,2,3,4,5,6,7,8,9\}$. Las teclas * y # están prohibidas.

Definición del subproblema

Sea $f(k, d)$ = número de secuencias de longitud k que comienzan en el dígito d .

Subestructura óptima

Decisión

Cuál es el segundo dígito: $v \in \text{vecinos}(d)$.

Subproblema generado

Contar las secuencias de longitud $k - 1$ que comienzan en v .

Demostración

Toda secuencia válida de longitud k comenzando en d tiene la forma $d \rightarrow s_2 \rightarrow \dots \rightarrow s_k$, donde $s_2 \in \text{vecinos}(d)$ y $s_2 \rightarrow \dots \rightarrow s_k$ es una secuencia válida de longitud $k - 1$ comenzando en s_2 . La correspondencia entre secuencias de longitud k desde d y los pares $(v, \text{secuencia de } k - 1 \text{ desde } v)$ con $v \in \text{vecinos}(d)$ es una biyección exacta. Cualquier error en $f(k - 1, v)$ se propaga directamente a $f(k, d)$.

Subproblemas traslapados

El árbol de recursión de $f(n, d)$ tiene hasta $|\text{vecinos}(d)| \leq 5$ hijos por nodo, generando hasta 5^{n-1} hojas. Sin embargo, los estados distintos son solo $n \times 10$.

Ejemplo

$f(n, 5)$ requiere $f(n - 1, 2)$, $f(n - 1, 4)$, $f(n - 1, 5)$, $f(n - 1, 6)$, $f(n - 1, 8)$. A su vez, $f(n, 2)$, $f(n, 4)$, $f(n, 6)$ y $f(n, 8)$ también requieren $f(n - 1, 5)$, por lo que ese subproblema es invocado múltiples veces. Sin memoización: $O(5^n)$; con programación dinámica: $O(10n)$.

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos



Relación de recurrencia

$$f(k, d) = \sum_{v \in \text{vecinos}(d)} f(k-1, v) \quad k > 1$$

$$\text{Total}(n) = \sum_{d \in D} f(n, d)$$

Verificación para $n = 2$ (esperado: 36):

	$d = 0$	$d = 1$	$d = 2$	$d = 3$	$d = 4$	$d = 5$	$d = 6$	$d = 7$	$d = 8$	$d = 9$	Total
1	1	1	1	1	1	1	1	1	1	1	10
2	2	3	4	3	4	5	4	3	5	3	36

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos

Inciso c



ALGORITMO CONTAR_COMBINACIONES(n)

vecinos $\leftarrow$ {

0:[0,8], 1:[1,2,4], 2:[2,1,3,5], 3:[3,2,6],

4:[4,1,5,7], 5:[5,2,4,6,8], 6:[6,3,5,9],

7:[7,4,8], 8:[8,5,7,9,0], 9:[9,6,8]

}

PARA d $\leftarrow$ 0 HASTA 9 HACER

dp[d] $\leftarrow$ 1

FIN PARA

PARA k $\leftarrow$ 2 HASTA n HACER

PARA d $\leftarrow$ 0 HASTA 9 HACER

dp_nuevo[d] $\leftarrow$ 0

FIN PARA

PARA d $\leftarrow$ 0 HASTA 9 HACER

PARA cada v EN vecinos[d] HACER

dp_nuevo[d] $\leftarrow$ dp_nuevo[d] + dp[v]

FIN PARA

FIN PARA

$dp \leftarrow dp_nuevo$

FIN PARA

$total \leftarrow 0$

PARA $d \leftarrow 0$ HASTA 9 HACER

$total \leftarrow total + dp[d]$

FIN PARA

RETORNAR $total$

FIN ALGORITMO

Complejidad temporal

En cada una de las $n - 1$ iteraciones se procesan 10 dígitos con a lo sumo 5 vecinos: $O((n - 1) \times 10 \times 5) = O(n)$.

Complejidad espacial

Solo dos arreglos de tamaño 10: $O(1)$.

Universidad del Valle de Guatemala

Facultad de Ingeniería

Análisis y diseño de algoritmos



Tabla de resultados

	Total de combinaciones
1	10
2	36
3	138
5	2,062
10	1,806,282